

22. Phase Prompts Pack

Ready-to-paste prompt patterns for Claude to implement, review, repair, and package work on QuantPipeline

1. How to use this prompt pack

Each prompt here is written as a reusable operating prompt for Claude. It assumes Claude has access to the codebase and the 01–20 documents. Replace bracketed placeholders only; keep the control language intact so work remains bounded and evidence-driven.

Prompt Pack Coverage Map

	Prompt type	Used when	Expected output
Implementation phase	Build against a bounded phase plan	New feature/spec delivery	Diff summary, changed files, tests, s
Critical review phase	Challenge claims and hidden gaps	Need skeptical verification	Truth report with PASS/PARTIAL/FAIL
Bugfix / hardening phase	Repair a bounded defect list	Failures already known	Patch summary, rerun results, residu
Release packaging phase	Freeze and document delivery	Preparing handoff or release	Package order, evidence, operator n

2. Prompt A — bounded implementation phase

You are implementing the next bounded phase of QuantPipeline. Read the provided documents and the current repository before coding. Do not assume file names, tables, services, or routes. Verify them.

Goal:
[INSERT PHASE GOAL]

Required in-scope surfaces:
[INSERT FILES / MODULES / ROUTES / TABLES / SCREENS]

Non-negotiable rules:

1. Work as an evolution, not a revolution.
2. Keep scope bounded and name out-of-scope items.
3. Before coding, restate what already exists and what is missing.
4. After coding, produce:
 - exact files changed
 - implementation summary
 - tests and checks run
 - screenshots / renders / endpoint evidence as relevant
 - PASS / PARTIAL / FAIL against acceptance criteria
5. Never claim completion without proof.

Acceptance criteria:

[INSERT ACCEPTANCE GATES]

3. Prompt B — skeptical review pass

You are not implementing now. You are acting as a skeptical reviewer. Challenge the claimed outcome against the code, artifacts, screenshots, test outputs, and live behavior if available.

Required output:

- Scope actually reviewed
- Evidence actually inspected
- What is confirmed working
- What is partially working
- What is unverified
- Hidden contradictions or drift
- PASS / PARTIAL / FAIL with reasons
- Exact bounded next actions

4. Prompt C — bugfix and hardening phase

You are repairing a bounded defect list in QuantPipeline.

Defect list:

[INSERT DEFECTS]

Rules:

- Reproduce each defect before fixing when feasible.
- Keep fixes surgical.
- Do not broaden scope into unrelated refactors.

- After each fix cluster, rerun the smallest relevant verification harness.
- Produce a defect-by-defect closure table with status, evidence, and residual risk.

5. Prompt D — document-driven build phase

You are implementing directly from the QuantPipeline specification package. Use documents [INSERT NUMBERS] as the source of truth, but verify the current repository before touching code.

Your job:

- map document requirements to existing code surfaces
- identify exact gaps
- implement only the requirements requested for this phase
- maintain naming consistency across product, architecture, data model, and API contract
- produce an evidence pack that links document requirements to implementation outcomes

6. Prompt E — release packaging and handoff

You are preparing a release-quality handoff.

Required outputs:

- ordered package structure
- updated README / handoff file
- exact deployment steps if relevant
- known limitations and unresolved items
- operator notes for the next model or engineer
- no packaging theater: if proof is missing, say so plainly

7. Prompt F — UI/UX fidelity phase

You are implementing a bounded UX/UI phase against QuantPipeline docs 16–20.

Required behavior:

- respect IA, flows, states, and component rules
- use existing tokens and patterns when present
- show key desktop/mobile/admin states
- verify not only appearance but also information hierarchy, trust cues, and action clarity
- return screenshots or rendered proof for the main scenarios

8. Prompt G — data/API consistency audit

You are auditing consistency across documents 10–12 and the current codebase.

Check:

- entity names
- field names and types
- enum semantics
- endpoint names and payload structure
- error semantics
- ownership of derived metrics and recommendation objects

Return a contradiction log and a repair plan ranked by severity.

9. Prompt H — high-trust continuation after a failed attempt

A previous attempt partially completed the task or crashed. Continue from the current state without pretending prior steps succeeded fully.

Before coding:

- list artifacts that actually exist
- list artifacts that were only claimed
- choose the next smallest safe sub-task
- continue without asking for confirmation between every sub-step

At the end:

- say exactly what was completed in this pass
- say what still requires another pass

10. Operator guidance

When to use	Best prompt
New feature phase from docs	Prompt A or D
Need skeptical truth report	Prompt B
Known defect cleanup	Prompt C
Preparing a package for another model or engineer	Prompt E
UX/UI refinement	Prompt F
Cross-layer contract checking	Prompt G
Recovery after partial crash or broken run	Prompt H